Karunya INSTITUTE OF TECHNOLOGY AND SCIENCES
(Declared as Deemed to be University under Sec.3 of the UGC Act, 1956)
A CHRISTIAN MINORITY RESIDENTIAL INSTITUTION
AICTE Approved & NAAC Accredited

SKILL BASED EVALUATION REPORT

(III Internal Assessment)

For

25AI2011

IMMERSIVE TECHNOLOGIES AND EXTENDED REALITY SYSTEMS

SUBMITTED BY

DEMIAN SHARWIN (URK23CS7011)

Nithya Maria Mathew (URK23CS7004)

G Vasu (URK23CS7130)

DIVISION OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY

OCTOBER 2026

AR Target Shooter:

An Augmented Reality Mobile Game Using Unity 6, AR Foundation and Google ARCore

Submitted By

DEMIAN SHARWIN (URK23CS7011)

Nithya Maria Mathew (URK23CS7004)

G Vasu (URK23CS7130)



DIVISION OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

KARUNYA INSTITUTE OF TECHNOLOGY AND SCIENCES (Declared as Deemed-to-be-under Sec-3 of the UGC Act, 1956) Karunya Nagar, Coimbatore - 641 114. INDIA

OCTOBER 2026

Karunya INSTITUTE OF TECHNOLOGY AND SCIENCES
(Declared as Deemed to be University under Sec.3 of the UGC Act, 1956)
A CHRISTIAN MINORITY RESIDENTIAL INSTITUTION
AICTE Approved & NAAC Accredited

DIVISION OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY

BONAFIDE CERTIFICATE

This is to certify that the skill-based evaluation report for the course 25AI2011 Immersive Technologies and Extended Reality Systems is a bonafide work done during the Odd semester of the academic year 2026-2027 by

DEMIAN SHARWIN (URK23CS7011)

Nithya Maria Mathew (URK23CS7004)

G Vasu (URK23CS7130)

in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering of Karunya Institute of Technology and Sciences.

Submitted for the assessment held on :29-10-26

Signature of the Class Teacher
Dr. Kavitha Margret Assistant Professor Division of Artificial Intelligence and Machine Learning

DECLARATION

I hereby declare that the skill-based evaluation report for the course 25AI2011 Immersive Technologies and Extended Reality Systems submitted by me to Karunya Institute of Technology and Sciences is an authentic work carried out by me under the guidance of Dr. Kavitha Margret. All the information presented in this report is original and has not been submitted for any other academic purpose.

I acknowledge that the sources of information used in this report have been appropriately cited and referenced. Any contributions from other individuals or organizations have been duly acknowledged.

I understand that any act of plagiarism or academic dishonesty is strictly prohibited and may result in disciplinary action as per the university's regulations.

I take full responsibility for the content and integrity of this report, and I am willing to provide further clarification or information if required.

Signature of the student

DIVISION OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY

PROJECT EVALUATION SHEET

S. No.	Rubrics	Maximum Marks	Marks Obtained
1	Certification	10	
2	Immersive Technologies Project	30	
	Total	40	

Project Evaluation

S. No.	Rubrics	Max. Marks	Marks Obtained
1	Immersive Experience Design (UI/UX)	5	
2	3D Modelling, Environment Design and Interaction	5	
3	XR Implementation and Technical Integration	5	
4	Interactivity, Navigation and User Engagement	4	
5	Innovation and Critical Thinking	4	
6	Performance, Testing and Optimization	3	
7	Presentation and Viva Voce	3	
8	Completeness of the Project Report	1	
	Total	30	

Signature of the Class Teacher
Dr. Kavitha Margret
Assistant Professor
Division of Artificial Intelligence and Machine Learning

TABLE OF CONTENTS

S.No	Content Title	Page No
1	**Abstract**	**1**
2	**Introduction**	**2**
3	**Problem Statement**	**3**
3.1	Project Objective	3
3.2	Specific Objectives	3
3.3	Game Concept and Scope	4
4	**Methodology / Architecture**	**5**
4.1	Tools and Technologies	5
4.2	System Architecture	5
4.3	Immersive Workflow	7
4.4	Game State Design	8
4.5	Development Methodology	8
5	**Implementation**	**10**
5.1	Game Setup and Configuration	10
5.2	Gameplay Implementation and Output Screenshots	12
5.3	Testing, Results and Final Game Screenshot	17
5.4	Problems Encountered and Solutions	19
6	**Conclusion**	**21**
7	**References**	**22**

1. ABSTRACT

AR Target Shooter is a skill-based augmented reality (AR) mobile game developed with Unity 6.3 LTS (6000.3.26f1), AR Foundation 6.3.5 and the Google ARCore XR Plugin. The game uses the rear camera of an ARCore-supported Android smartphone to show the player's real surroundings, detects real floors and walls, and then fills the surrounding space with floating bullseye targets. The player taps the touchscreen to launch a physics-based projectile from the camera towards the touched point. Gravity bends the path of each shot into a parabola, so the player must judge distance as well as direction.

A hit on the outer ring scores 10 points and a hit on the centre zone (within 5 cm of the target centre) scores 25 points. Each round lasts 60 seconds and can be played at three difficulty levels (Easy, Normal and Hard) that change the number of simultaneous targets, how long a target lives, and how quickly a replacement appears. The project is written in C# with the Universal Render Pipeline (URP), the Unity Input System (Enhanced Touch), TextMeshPro for the interface and a Particle System for hit feedback.

The application was built in seven verified phases: project set-up, plane detection, target spawning, projectile shooting, hit handling and scoring, the complete game loop (menu, timer, game over, best score), and final polish (pause and resume, permission and availability handling, clean-up and release settings). Each phase was compiled, installed as an APK on a Samsung Galaxy S23 Ultra, tested on the device and committed to Git. In the device tests the game ran at about 30 frames per second, and a typical Normal-difficulty round produced a final score of 205 from 16 hits at 80% accuracy. This report describes the system architecture, the configuration, the gameplay implementation, the test results with on-device screenshots, the problems that were solved, and the limitations that remain.

2. INTRODUCTION

Augmented reality places computer-generated objects into a live view of the real world. On a modern smartphone this is achieved by combining the camera image with motion-sensor data so that the device can estimate its own position and orientation, and by recognising flat surfaces such as floors, tables and walls. Virtual objects can then be positioned relative to the real environment and stay in place as the user moves. Google ARCore provides this tracking on Android devices, and Unity's AR Foundation framework exposes it through a single cross-platform API.

Shooting-gallery games are a natural fit for AR. The player stands in a real room, targets float in the air around them, and the physical movement of the phone becomes the aiming control. The game design is easy to learn, because the player only needs to point and tap, yet it exercises several core ideas of immersive technology: environment understanding, spatial placement of virtual content, ray-based interaction, physics simulation and real-time feedback.

This project, **AR Target Shooter**, was inspired by a five-step concept: (1) detect the environment, (2) generate targets, (3) create a projectile, (4) aim using touch, and (5) detect hits. Each step maps to a specific Unity and AR Foundation feature, as shown in Table 2.1.

Table 2.1: Mapping of the concept steps to the technology used

Step	What happens	Unity / AR feature
1. Detect environment	The phone finds floors and walls and shows where a surface is	ARPlaneManager, ARRaycastManager, reticle
2. Generate targets	Bullseye targets appear in the air around the player	Prefab, TargetSpawner, random placement in an arc
3. Create projectile	A tap launches a small sphere that falls under gravity	Rigidbody, ProjectilePool, ContinuousDynamic collision
4. Aim using touch	The tapped screen point decides the shot direction	Enhanced Touch, ScreenPointToRay
5. Detect hits	Collisions award points and play an effect	SphereCast, layers, ScoreManager, Particle System

The remainder of the report states the problem and objectives (Section 3), explains the methodology and architecture with diagrams (Section 4), documents the implementation with code excerpts and device screenshots (Section 5), and closes with conclusions and references.

3. PROBLEM STATEMENT

Most casual mobile games are played entirely on a flat screen with little connection to the player's surroundings. A short, accessible game that uses the real room as its play area can make practice of aiming, timing and hand-eye coordination more engaging. Building such a game on a phone raises a set of engineering challenges: the AR session must be started and kept tracking, real surfaces must be detected so that virtual content can be placed sensibly, targets must be spawned around the player without overlapping, touch input must be turned into a believable physical shot, fast projectiles must not pass through small targets, and the whole experience must stay responsive on mobile hardware.

This project addresses these challenges by designing, implementing and testing a complete AR target-shooting game, including a menu, scoring, timer, game-over screen and error handling, and by packaging it as an Android application for on-device evaluation.

3.1 Project Objective

To design, implement and test a complete augmented reality target-shooting game in Unity in which floating targets are generated in the player's real environment, projectiles are launched by touch and follow realistic projectile motion, collisions are detected reliably, and a score is awarded within a timed round.

3.2 Specific Objectives

1. Configure Unity 6.3 LTS, URP, AR Foundation and ARCore to run an AR session on a compatible Android device.
2. Detect horizontal and vertical planes and show a placement reticle where a ray from the screen centre meets a plane.
3. Create a reusable bullseye target prefab and spawn a controlled number of targets 1.5 to 4 m from the player, at least 0.7 m apart.
4. Convert a screen tap into a camera ray and launch a physics-based projectile with gravity from an object pool.
5. Detect hits reliably with continuous collision detection and a sphere-cast, using layers so that shots ignore real-world planes.
6. Award 10 points for an outer-ring hit and 25 points for a centre hit, and show a burst effect and sound.
7. Provide a complete game loop: menu, three difficulty levels, a 60-second timer, a game-over screen and a stored best score.

8. Handle app pause and resume, missing camera permission and unsupported devices gracefully.
9. Build an Android APK and verify behaviour on a Samsung Galaxy S23 Ultra with documented results.

3.3 Game Concept and Scope

Table 3.1: Game concept and scope

Item	Details
Game title and genre	AR Target Shooter; casual, skill-based augmented reality shooting-gallery mobile game.
Target platform	Android phone with Google ARCore support. Tested on a Samsung Galaxy S23 Ultra (SM-S918B).
Intended player and goal	A single player who aims at floating bullseye targets and taps to shoot as many as possible within 60 seconds.
Core controls	Touchscreen tap: a ray is cast from the AR camera through the touch point and a projectile is launched along it.
Core mechanics	Plane detection, target spawning and respawning, projectile physics, sphere-cast hit detection, scoring, timer, difficulty presets.
Success feedback	Target disappears with an orange particle burst and a pop sound; the score increases; centre hits play a brighter two-tone sound.
Scope and constraints	Single-scene game, OpenGL ES 3 only, portrait orientation. Multiplayer, online leaderboards, moving or bonus targets and a Play Store release are future work.

4. METHODOLOGY / ARCHITECTURE

The project follows an incremental, phase-by-phase game-development workflow. The game was divided into seven phases. In each phase the code was written (with assistance from the Claude Code AI coding tool, guided by a project rules file named CLAUDE.md), compiled in Unity, installed on the phone, tested against a written checklist, and only then committed to Git. This kept every phase small enough to debug and ensured that the repository history always represented a version that had been verified on a real device.

4.1 Tools and Technologies

Table 4.1: Tools and technologies

Component	Technology / Configuration	Role
Game engine	Unity 6.3 LTS (6000.3.26f1)	Scene creation, physics, object lifecycle and Android build.
Render pipeline	Universal Render Pipeline (URP)	Rendering of virtual objects over the camera background.
AR framework	AR Foundation 6.3.5	Cross-platform AR session, plane detection and raycasting.
AR plug-in	Google ARCore XR Plugin	Motion tracking and environment understanding on Android.
Input	Unity Input System 1.20.0 (Enhanced Touch)	Reading touchscreen taps through a shared TouchInputService.
Physics	Rigidbody, SphereCast, layer collision matrix	Projectile motion and hit detection.
User interface	Canvas, TextMeshPro, Input System UI module	Menu, HUD and game-over panels inside the screen safe area.
Effects and audio	Particle System, AudioSource	Pooled hit burst and generated hit sounds.
Language and editor	C#, Visual Studio Code	Gameplay and editor scripts.
Scripting backend	IL2CPP, ARM64, OpenGLES3	Android player build.
Version control	Git	One commit per verified phase.
Target device	Samsung Galaxy S23 Ultra (SM-S918B)	APK installation and on-device testing.

4.2 System Architecture

The game is organised into four cooperating layers (Figure 4.1). Scripts in different layers do not hold references to each other's internals; they communicate through public methods and C#

events, which keeps each script small and allows a phase to be changed without breaking the others.

1. **AR and input layer:** the AR Session and XR Origin supply the camera pose; ARPlaneManager detects surfaces; ARRaycastManager finds the point on a plane under the reticle; TouchInputService reference-counts Enhanced Touch so that several scripts can share it safely.
2. **Gameplay layer:** TargetSpawner creates and respawns targets; ShootingController turns taps into projectiles drawn from ProjectilePool; ScoreManager awards points; HitEffects plays the burst and sound.
3. **Game-flow layer:** GameManager runs the state machine (Menu, Playing, Paused, Game Over), the timer and the difficulty presets; ARAvailabilityMonitor watches the AR session for unsupported devices and denied camera permission.
4. **Presentation layer:** GameUI builds the menu, HUD and game-over panels with TextMeshPro and applies the device safe area.

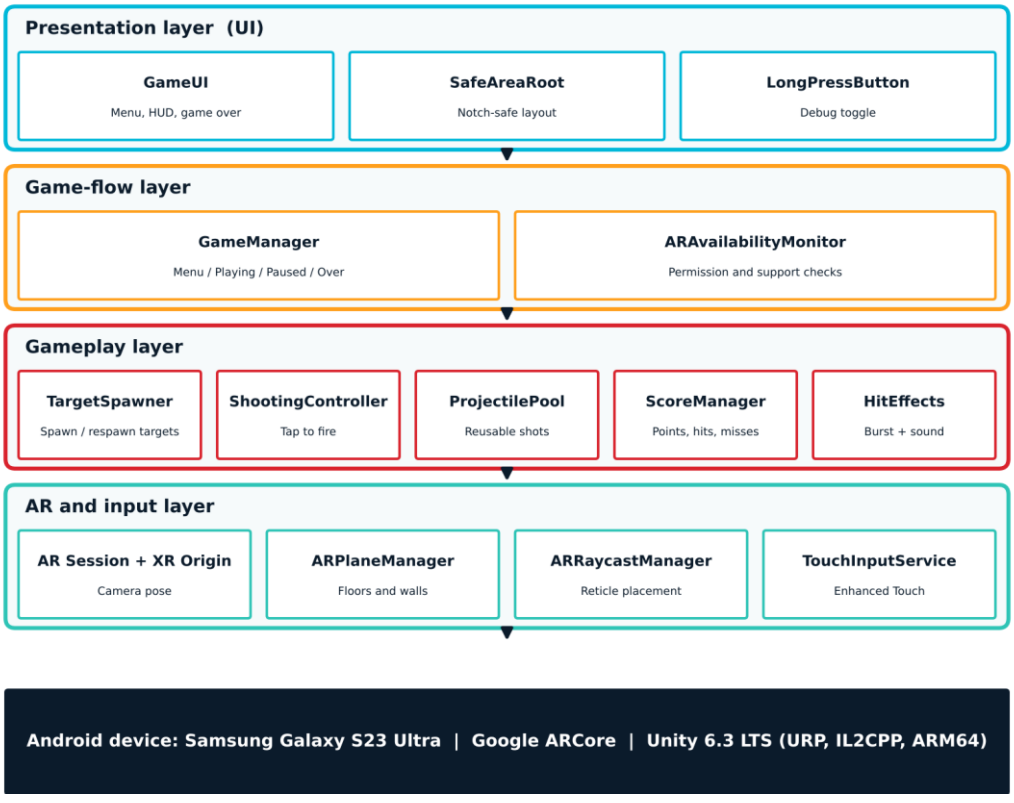


Figure 4.1: Layered system architecture of AR Target Shooter

Figure 4.2 shows how a plane found by ARCore becomes a reticle position that tells the player where AR tracking is working, and how targets float in the same space.

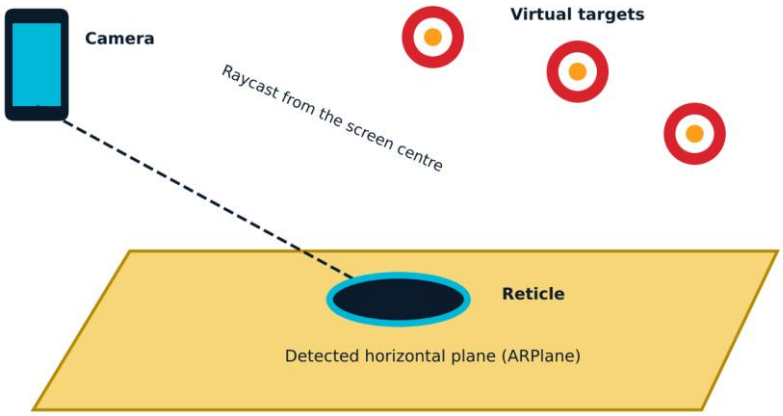


Figure 4.2: Plane detection, raycast from the screen centre and the placement reticle

4.3 Immersive Workflow

Figure 4.3 summarises the two main flows of the running game: the player-interaction flow, from a finger tap to a score, and the target-spawning flow, from a detected plane to a respawned target.

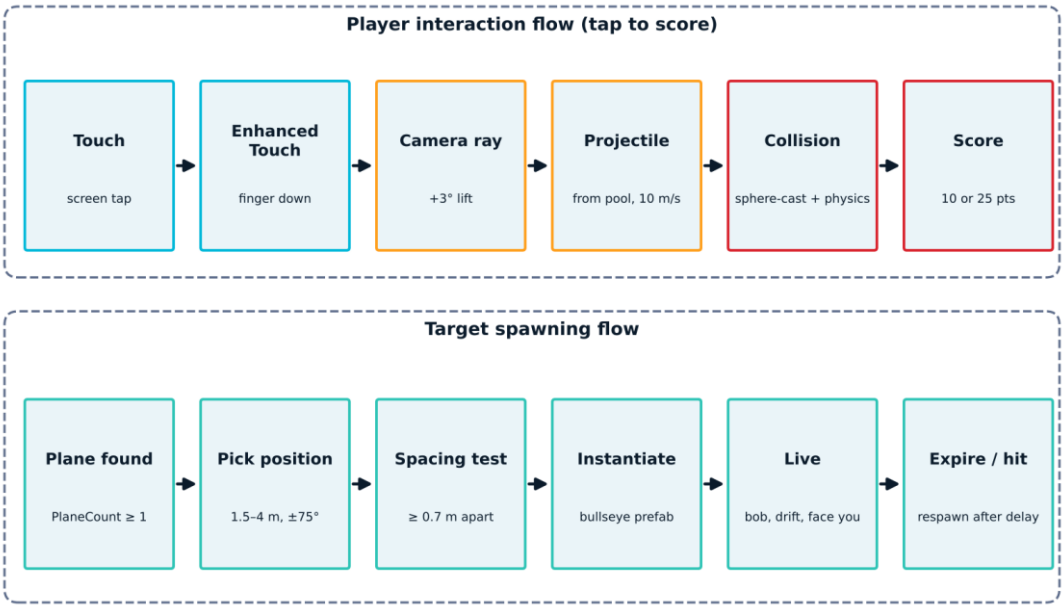


Figure 4.3: Player interaction flow and target spawning flow

The immersive workflow at run time is:

1. The application launches. ARCore requests camera permission, and ARAvailabilityMonitor reports a plain-language message if the device is unsupported or permission is denied.
2. ARCore starts tracking the device position and orientation and begins detecting horizontal and vertical planes. The menu shows "Scan a surface to begin" until at least one plane exists, then "Ready!".

3. The player selects a difficulty and presses Start. Planes and the reticle are hidden, the 60-second timer starts, and TargetSpawner places the first targets.
4. Each target is placed 1.5 to 4 m from the camera inside a +/-75 degree arc, bobs and drifts slowly, always turns to face the player, and respawns elsewhere when it is hit or expires.
5. When the player taps the screen, a ray is cast through the touch point, tilted 3 degrees upward, and a pooled projectile is launched at 10 m/s with gravity enabled.
6. A hit is detected by physics collision and by a sphere-cast between physics steps. ScoreManager awards 10 or 25 points, the target is despawned, and HitEffects plays a burst and a pop.
7. When the timer reaches zero, firing and scoring are disabled, targets are cleared, and the game-over panel shows the score, hits, accuracy and best score.

4.4 Game State Design

A small state machine in GameManager controls what is active at any time (Figure 4.4). Firing, spawning and scoring are enabled only in the Playing state. If the app goes to the background during a round, the game enters the Paused state, which stops spawning and clears targets but keeps the score, shot count and remaining time. The round resumes only when the AR session is tracking again and at least one plane exists. Timers use Time.deltaTime and the game never changes Time.timeScale, so pausing does not freeze unrelated systems.

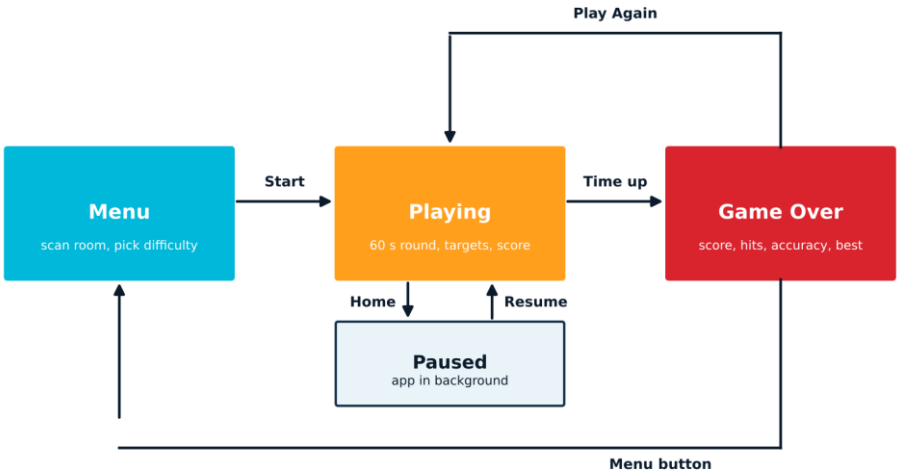


Figure 4.4: Game state machine

4.5 Development Methodology

Figure 4.5 shows the seven phases. Each phase ended with an on-device test and a Git commit, and the project rules file CLAUDE.md was updated with what had been verified. Editor-only builder scripts (for the target prefab, projectile prefab, hit effects and game UI) create assets from menu commands so that the build never depends on runtime shader lookups.

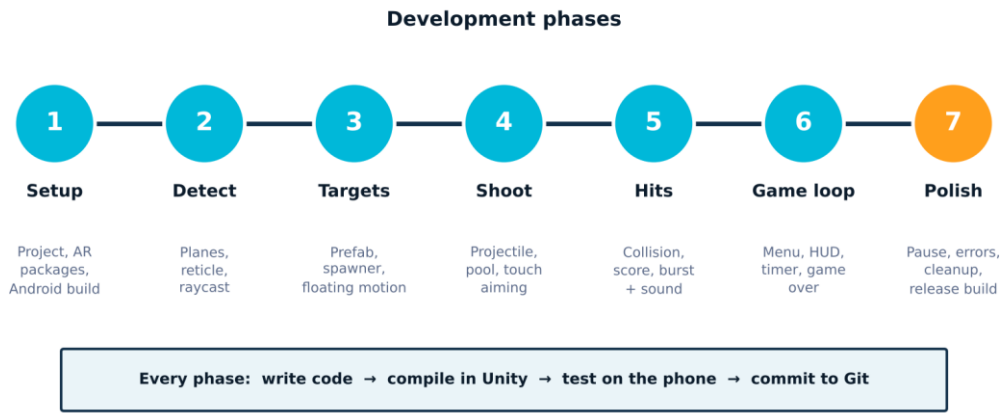


Figure 4.5: The seven development phases

Table 4.2: Summary of what each phase delivered

Phase	Delivered	Verified on device
1 Setup	Unity 6.3 project, URP, AR Foundation, ARCore, Android build, debug overlay	Camera feed, AR session tracking, 30 FPS
2 Detect	PlaneDetectionController, PlacementReticle, temporary tap-to-place marker	Planes shown, reticle "valid pose"
3 Targets	Target prefab builder, Target, TargetSpawner	Five floating targets, respawn
4 Shoot	Projectile, ProjectilePool, ShootingController, layer matrix	Projectiles arc and pass through planes
5 Hits	ScoreManager, HitEffects, generated sounds	Score, hits and misses update
6 Game loop	GameManager, GameUI, difficulty, timer, game over, best score	Full round from menu to game over
7 Polish	Pause and resume, availability monitor, clean-up, release settings	Pause and resume work

5. IMPLEMENTATION

The implementation uses a single Unity scene, Main, that contains the AR Session, XR Origin with its AR camera, and a set of manager objects: TargetSpawner, ShootingController, ScoreManager, HitEffects, GameManager and the game canvas. Targets and projectiles are stored as prefabs so that the spawner and the pool can create many instances during play. All gameplay scripts live under Assets/_Project/Scripts in the folders Core, AR, Gameplay, UI and Editor.

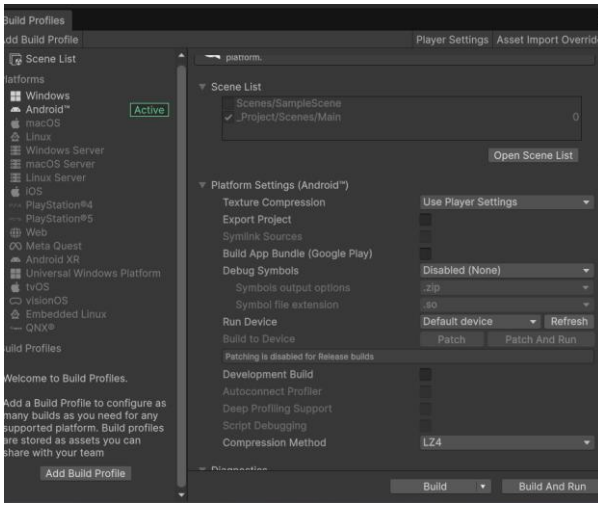
5.1 Game Setup and Configuration

5.1.1 Project Creation and Android Platform

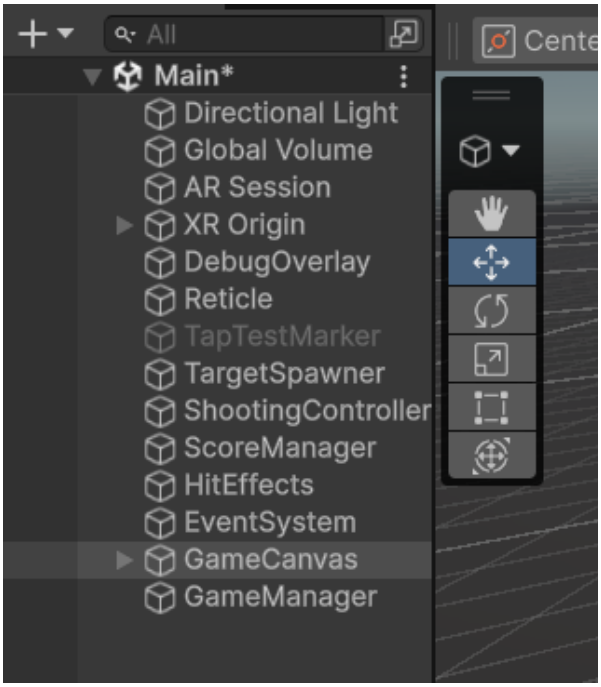
A Unity 6.3 LTS project named ARTargetShooter was created from the URP template. Android Build Support (with OpenJDK, Android SDK and NDK) was installed through Unity Hub, and the active build platform was switched to Android through File, Build Profiles. AR Foundation 6.3.5, the Google ARCore XR Plugin and the Input System package were installed through the Package Manager. The project was initialised as a Git repository and a CLAUDE.md rules file was added to record the conventions, such as always using the TouchInputService and never calling Shader.Find at run time.

5.1.2 XR and Scene Configuration

- Enabled Google ARCore in Project Settings, XR Plug-in Management (Android tab).
- Set Graphics API to OpenGLES3 only, scripting backend to IL2CPP, target architecture to ARM64 and the minimum API level to one supported by ARCore.
- Locked the screen orientation to Portrait so that rotating the phone cannot change the layout mid-round.
- Replaced the default camera with AR Session and XR Origin (Mobile AR), and added the AR Background Renderer Feature to the URP renderer so that the live camera image is drawn behind virtual objects.
- Added ARPlaneManager and ARRaycastManager to the XR Origin.



Build Profiles: Android active, Main scene enabled



Final scene hierarchy of Main

Figure 5.1: Unity Build Profiles window and the final scene hierarchy

5.1.3 Prefabs, Layers and Materials

The target prefab is a bullseye built from three thin cylinders with the materials Target_Red, Target_White and Target_Center, using the URP Unlit shader. A custom editor command (ARTargetShooter, Create Target Prefab) builds the materials and the prefab, and adds the Target layer (layer 8). A second command creates the projectile prefab: an unscaled root with a 0.04 m radius SphereCollider and a Rigidbody (mass 0.05) and a child visual sphere of 0.08 m, on the Projectile layer (layer 9). Using editor-created material assets guarantees that the shaders are included in the build.

The layer collision matrix disables Projectile against Default and against Projectile, but keeps Projectile against Target enabled. This is what allows shots to fly straight through the invisible AR plane colliders while still hitting targets.

5.1.4 Scene Wiring

Each manager is an empty GameObject with one script. TargetSpawner receives the Target prefab, the AR camera and the plane controller. ShootingController receives the Projectile prefab, the AR camera and the Target mask. ScoreManager receives the ShootingController. HitEffects receives the ScoreManager, the HitBurst prefab and two audio clips. GameManager receives references to all of them and to GameUI. These references are essential: an unassigned prefab prevents spawning, and an unassigned camera prevents correct spawning and aiming. A Verify Setup command for each phase logs OK or MISSING for every required asset.

5.1.5 Build and Installation

The application was built as an Android APK from Build Profiles and installed on the Samsung Galaxy S23 Ultra with Build And Run. For release, the package name was set to com.demian.artargetshooter, the product name to AR Target Shooter, the version to 1.0.0 and managed code stripping to Medium. The URP Unlit and Lit shaders were added to Always Included Shaders after a stripped-shader problem (Section 5.4).

5.2 Gameplay Implementation and Output Screenshots

The code excerpts below are condensed from the project scripts to show the core logic; helper code, null checks and logging are omitted.

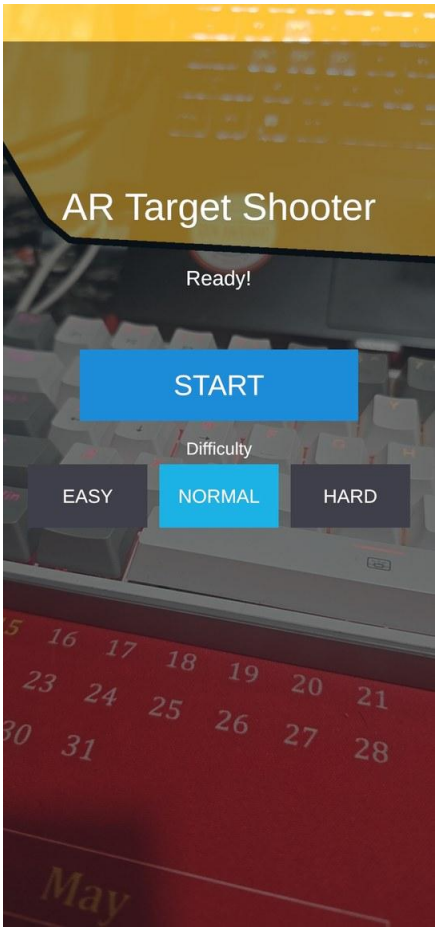
5.2.1 Plane Detection and Reticle: PlaneDetectionController.cs, PlacementReticle.cs

```
planeManager.requestedDetectionMode =
    PlaneDetectionMode.Horizontal | PlaneDetectionMode.Vertical;
planeManager.trackablesChanged.AddListener(OnPlanesChanged);

void OnPlanesChanged(ARTrackablesChangedEventArgs<ARPlane> args)
{
    foreach (var p in args.added)   planeIds.Add(p.trackableId);
    foreach (var p in args.removed) planeIds.Remove(p.Key);
    PlaneCount = planeIds.Count;     // set of ids, not +1 / -1
    OnPlaneCountChanged?.Invoke(PlaneCount);
}

// PlacementReticle.Update()
var centre = new Vector2(Screen.width / 2f, Screen.height / 2f);
if (raycastManager.Raycast(centre, hits,
        TrackableType.PlaneWithinPolygon))
{
    transform.SetPositionAndRotation(hits[0].pose.position,
                                     hits[0].pose.rotation);
    ring.SetActive(true);
}
else ring.SetActive(false);
```

Plane detection is enabled for both horizontal and vertical surfaces. In AR Foundation 6 the trackablesChanged event reports added, updated and removed planes. The controller stores plane identifiers in a set, so a duplicate event cannot make the count wrong. Every frame the reticle casts a ray from the screen centre against plane polygons; it moves to the hit point and hides when there is no hit. The count of planes gates the Start button in the menu.



Menu with Ready status and difficulty buttons



Detected planes (yellow) and the reticle (cyan ring)

Figure 5.2: Menu screen and plane detection with the placement reticle on the Galaxy S23 Ultra

5.2.2 Target Spawning: TargetSpawner.cs and Target.cs

```
Vector3 PickPosition()
{
    for (int i = 0; i < 15; i++)                // up to 15 attempts
    {
        float dist  = Random.Range(1.5f, 4f);     // metres
        float angle = Random.Range(-75f, 75f);    // degrees
        float h     = Random.Range(-0.4f, 1.2f); // vs camera
        Vector3 fwd = Vector3.ProjectOnPlane(
            cam.forward, Vector3.up).normalized;
        Vector3 pos = cam.position
                    + Quaternion.AngleAxis(angle, Vector3.up) * fwd * dist
                    + Vector3.up * h;
        pos.y = Mathf.Max(pos.y, lowestFloorY + 0.3f); // above floor
        if (FarEnoughFromOthers(pos, 0.7f)) return pos; // >= 0.7 m
    }
    return fallbackPosition;
}
```

Targets are spawned in front of the camera inside a 150 degree arc, so the player is never surrounded by targets they cannot see (Figure 5.3). A target is accepted only if it is at least 0.7 m from the others, and its height is clamped above the lowest upward-facing plane. Each Target moves in a figure-eight bob and drift using random parameters, turns around the Y axis to face the camera, scales in over about 0.3 s, lives for the preset lifetime, and then scales out. A single

refill coroutine tops the number of targets back up to the preset count after the respawn delay, staggering new targets by 0.25 s.

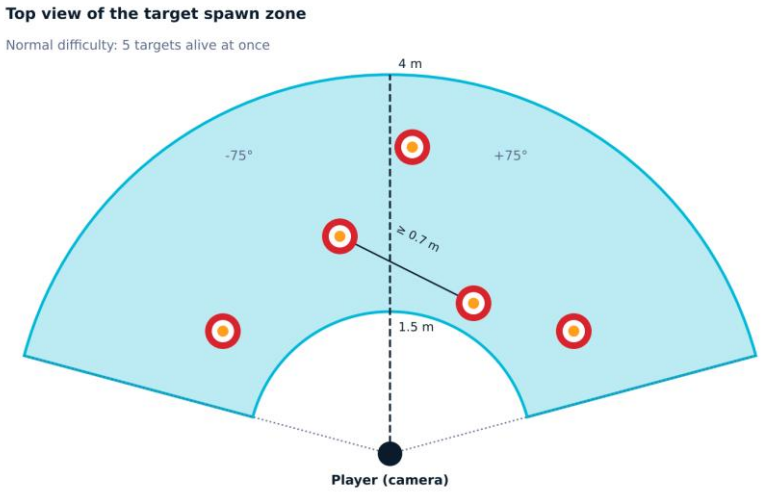


Figure 5.3: Top view of the target spawn zone (1.5 to 4 m, +/-75 degrees, at least 0.7 m apart)

5.2.3 Touch Aiming and Projectile: ShootingController.cs and Projectile.cs

```
// ShootingController: on finger down
if (Time.time < nextShotTime || IsOverUI()) return;  // 0.15 s cooldown
Ray ray = cam.ScreenPointToRay(fingerPosition);
// tilt the ray 3 degrees upward about the camera right axis
Vector3 dir = Quaternion.AngleAxis(-3f, cam.transform.right)
              * ray.direction;
Projectile p = pool.Get();
p.Launch(ray.origin, dir * launchSpeed);   // launchSpeed = 10 m/s
ShotsFired++;
nextShotTime = Time.time + 0.15f;

// Projectile
rb.collisionDetectionMode = CollisionDetectionMode.ContinuousDynamic;
rb.interpolation = RigidbodyInterpolation.Interpolate;

void FixedUpdate()
{
    Vector3 delta = rb.position - lastPos;
    if (!hasHit && Physics.SphereCast(lastPos, 0.04f,
            delta.normalized, out var hit,
            delta.magnitude, targetMask))
        RegisterHit(hit);          // one hit per shot
    lastPos = rb.position;
}
```

The tap position is converted into a camera ray with ScreenPointToRay. The ray is tilted upward by 3 degrees about the camera's right axis, and the projectile is launched at 10 m/s with ordinary gravity. Because the vertical position of a body under constant gravity follows y = x tan(theta) - g x squared / (2 v squared cos squared(theta)), the path is a parabola and a 4 m shot drops by about 0.8 m below the aim line (Figure 5.4). The player therefore learns to aim slightly above the target. Projectiles are recycled through a queue-based pool and are removed after 5 s or 30 m of travel. Taps that fall on UI buttons are ignored through an EventSystem check.

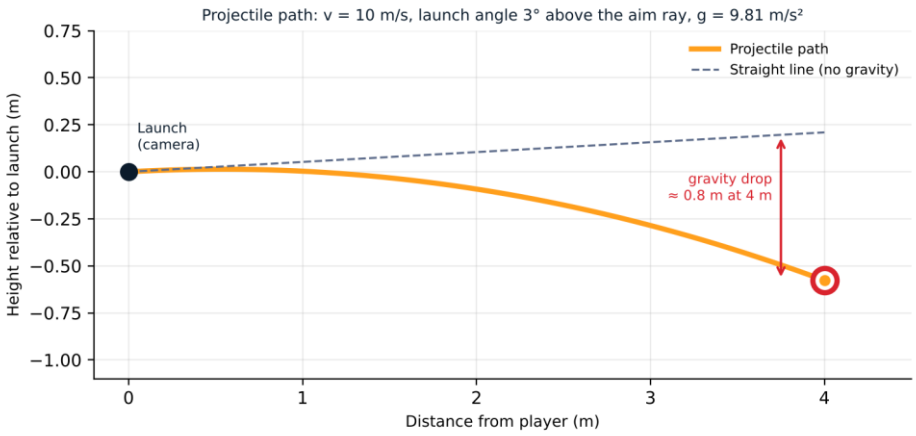


Figure 5.4: Computed projectile path for the default launch parameters

5.2.4 Hit Detection, Scoring and Effects: ScoreManager.cs and HitEffects.cs

```
void HandleTargetHit(Target t, Vector3 hitPoint)
{
    if (!scoringEnabled || t == null || !t.IsAlive)
        return;                              // no double hits
    // distance from the centre, in the target plane
    float r = Vector3.ProjectOnPlane(
        hitPoint - t.transform.position,
        t.transform.forward).magnitude;
    int points = (r <= centreRadius) ? 25 : 10;  // 0.05 m
    Score += points;  Hits++;
    t.Despawn(true);       // only place that despawns on a hit
    OnScoreChanged?.Invoke(Score);
    OnTargetScored?.Invoke(t.transform.position, points == 25);
}
```

Hits travel through events (Figure 5.5). Each pooled projectile raises OnHitTarget; ShootingController forwards it as OnTargetHit; ScoreManager checks that the target is still alive and awards the points; HitEffects listens to OnTargetScored and plays a pooled particle burst and a sound at the target's position at the moment of the hit. Misses are not counted separately: they are derived as shots fired minus hits. The scoring zones are shown in Figure 5.6.

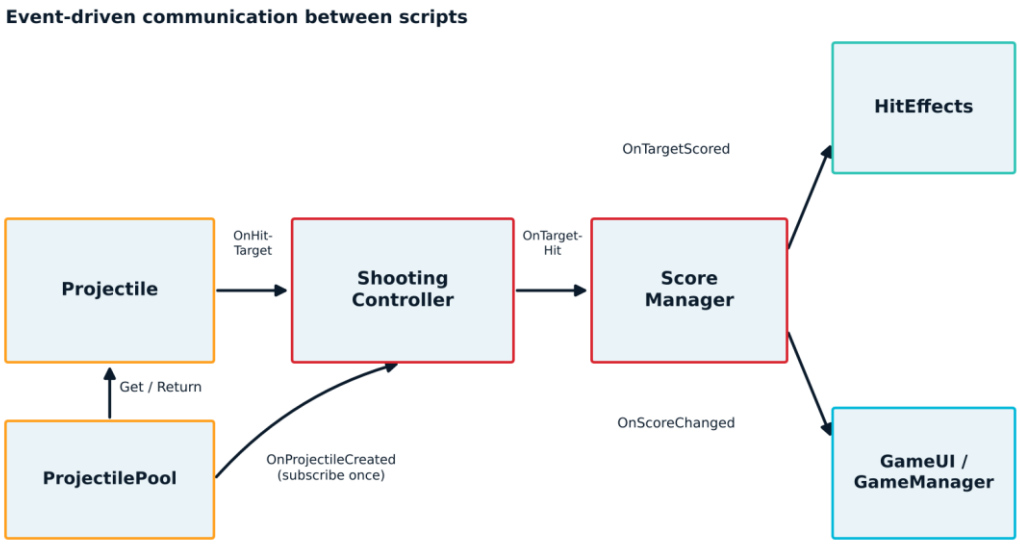


Figure 5.5: Event flow from projectile to score, effects and interface

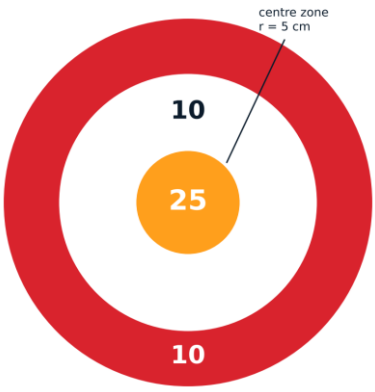


Figure 5.6: Scoring zones of a target (10 points outer ring, 25 points centre)

5.2.5 Game Loop and Difficulty: GameManager.cs and GameUI.cs

GameManager applies the selected difficulty to the spawner when a round starts (Table 5.1, Figure 5.7). StartGame stops and clears the spawner, resets score and shot counters, enables firing and scoring and starts spawning. EndGame disables firing and scoring first, so a late projectile cannot add score after time is up, then clears the targets, saves the best score in PlayerPrefs and raises the game-over event. GameUI shows the menu, HUD (score at the top left, time at the top right, turning red in the last 10 seconds) and the game-over panel, all inside Screen.safeArea so that the camera cut-out and gesture bar never hide information.

Table 5.1: Difficulty presets (round length 60 s for all)

Difficulty	Targets alive at once	Target lifetime (s)	Respawn delay (s)
Easy	4	25	1.2
Normal	5	20	1.0

Difficulty	Targets alive at once	Target lifetime (s)	Respawn delay (s)
Hard	7	12	0.6

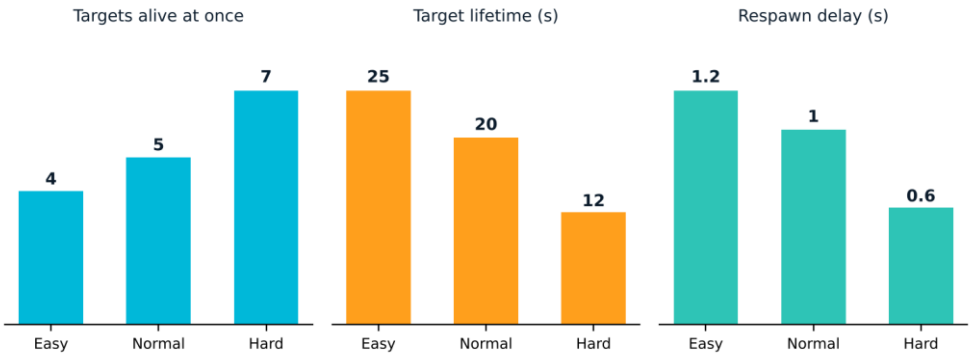


Figure 5.7: Difficulty presets

5.3 Testing, Results and Final Game Screenshot

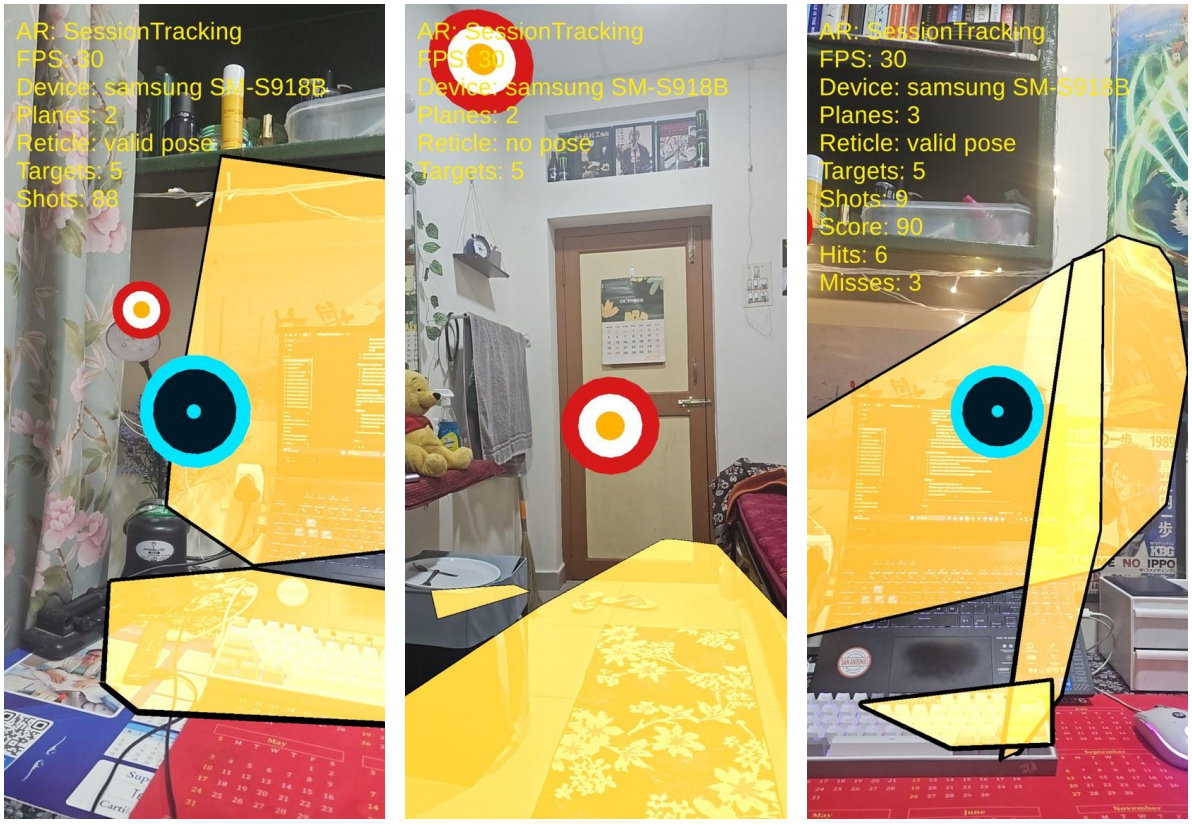
Testing was carried out on a Samsung Galaxy S23 Ultra (SM-S918B, Android) in an indoor room, using a development overlay that prints the AR session state, frames per second, plane count, reticle state, target count, shots, score, hits and misses. Every phase had a written checklist, and the results below are the outcomes observed on the device.

Table 5.2: Test cases and observed results

ID	Test	Expected behaviour	Observed on device	Result
T1	Launch and menu	Menu shows title, status text, Start and difficulty buttons	Menu appears over the camera view with Start and EASY / NORMAL / HARD; Normal highlighted	Pass
T2	Camera and AR tracking	Live rear camera; AR session tracking	Overlay reports AR: SessionTracking	Pass
T3	Plane detection	At least one plane found on a textured surface	Overlay showed 2 to 3 planes in the test room	Pass
T4	Reticle	Ring sits on the plane under the screen centre	Reticle: valid pose when aimed at a plane	Pass
T5	Target spawning	Five targets on Normal, 1.5 to 4 m away	Targets: 5 shown on the overlay; targets float and face the player	Pass
T6	Shooting	Tap fires a projectile that arcs; counter increases; shots ignore planes	Shots counter increased with each tap; shots flew through detected planes	Pass
T7	Hit and score	10 points ring, 25 points centre; misses = shots - hits	Score 90 after 9 shots: 6 hits (4 x 10 + 2 x 25) and 3 misses	Pass
T8	Round end	Game-over panel with score, hits, accuracy, best	Final Score 205, Hits 16, Accuracy 80%, Best 205, "New best!"	Pass
T9	Play Again and Menu	Clean restart with no leftover targets	Round restarted cleanly; Menu returned to the start screen	Pass

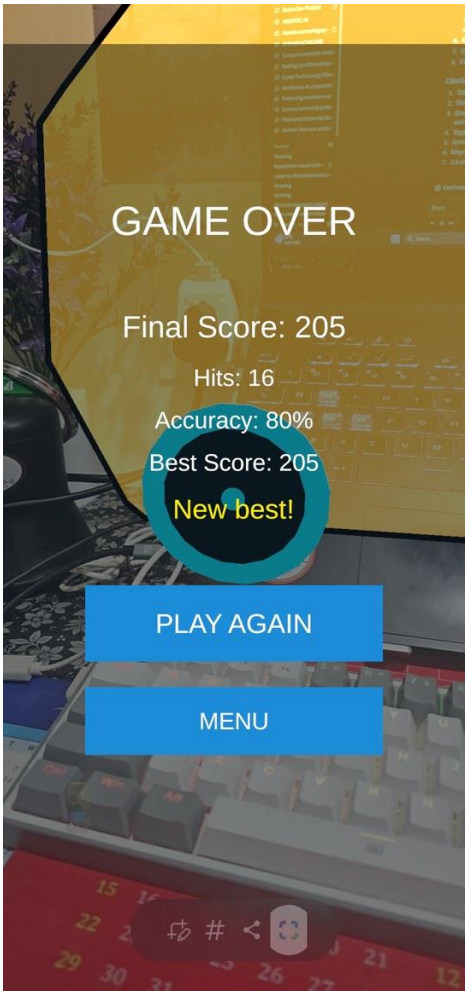
ID	Test	Expected behaviour	Observed on device	Result
T10	Pause and resume	Round survives sending the app to the background	Round paused and resumed once tracking recovered	Pass
T11	Frame rate	Smooth gameplay	About 30 FPS throughout (matches the camera rate of the device)	Pass
T12	Plane overlay on game over	Debug planes hidden outside the menu	Yellow plane overlay still visible on the game-over screen of the tested build	Cosmetic issue

The 20 shots of the best recorded round gave 16 hits, an accuracy of 80%, and a final score of 205. The centre zone is only 5 cm in radius, so most points came from outer-ring hits. The 30 FPS reading is set by the ARCore camera rate on the device, so the real cost of seven targets on Hard difficulty was not measured separately, and this is listed as a limitation.



Five targets with the reticle (Targets: 5, FPS: 30) | *A target floating in the real room* | *Mid-round: Score 90, Hits 6, Misses 3*

Figure 5.8: Gameplay on the Galaxy S23 Ultra: targets in the real room and live score read-out



Game-over panel: Final Score 205, Hits 16, Accuracy 80%, Best Score 205, "New best!"

Figure 5.9: Final game screenshot: game-over screen with Play Again and Menu buttons

5.4 Problems Encountered and Solutions

Table 5.3: Problems encountered and how they were solved

Problem	Cause	Solution
Targets and particles rendered pink / purple in the APK	A material shader was stripped from the build because it was only referenced at run time	Created materials as editor assets with the URP Unlit shader and added URP Unlit and Lit to Always Included Shaders; avoided Shader.Find at run time
Projectiles stopped on the floor and never reached targets	Detected AR planes carry colliders that shots hit first	Placed targets and projectiles on separate layers and disabled Projectile against Default and Projectile in the collision matrix
Fast shots passed through small targets	Discrete collision detection can skip a 0.18 m target between physics steps	Set ContinuousDynamic and interpolation, and added a SphereCast between FixedUpdate positions
One target could score twice or by two shots at once	Two projectiles can hit in the same step; a despawning target is still on screen for 0.3 s	Targets are hittable only while alive; ScoreManager re-checks IsAlive and is the only caller of Despawn(true)

Problem	Cause	Solution
Taps on buttons also fired shots; UI errors with the new Input System	No EventSystem, or the old Standalone Input Module was created	GameUIBuilder creates an EventSystem with the Input System UI Input Module and ShootingController ignores touches over UI
Misses counted shots from earlier rounds	ScoreManager.ResetScore did not reset the shot counter	Added ShootingController.ResetShots() and call it at the start of every round
Round state lost when the app was backgrounded	AR tracking is lost on pause	Added a Paused state that keeps score and time, and resumes only when the AR session is tracking and a plane exists
Blank screen if camera permission was denied or ARCore was unsupported	No handling of AR session availability	ARAvailabilityMonitor shows a message, offers Retry and opens app settings
Test-only assets left in the project	Temporary tap-to-place marker, sample scene and tutorial files from earlier phases	Removed in the Phase 7 clean-up after the scene references were cleared

6. CONCLUSION

AR Target Shooter demonstrates a complete mobile augmented reality game built with Unity 6.3 LTS, AR Foundation, Google ARCore, URP, the Unity Input System and C#. The game detects the real environment, spawns floating targets in the space around the player, converts touch input into physics-based projectiles, detects hits reliably with continuous collision detection and a sphere-cast, and rewards the player with a score, an effect and a stored best score within a timed round that offers three difficulty levels.

The project covers the practical tasks that make up immersive application development: configuring Android and XR support, setting up the AR session and camera background, creating and wiring prefabs, designing layers and collisions, building an interface that respects the device safe area, handling app pause and permission problems, and packaging the application for a real device. The phase-by-phase method, with a written test checklist and a commit after every verified phase, kept the project debuggable and produced the documented results of Section 5.3.

Limitations. The yellow plane overlay remained visible on the game-over screen of the tested build; the frame rate on Hard difficulty was not profiled separately; testing was done on one device; and the camera-permission and unsupported-device screens were implemented but not exercised on a clean install.

Future work. Possible extensions are a trajectory preview while aiming, moving and bonus targets, occlusion of targets behind real objects, an online leaderboard, haptic feedback, and a Google Play release with its own icon and signing key.

7. REFERENCES

1. Unity Technologies. Unity 6 Documentation. https://docs.unity3d.com/
2. Unity Technologies. AR Foundation Documentation. https://docs.unity3d.com/Packages/com.unity.xr.arfoundation@6.3/manual/index.html
3. Google. ARCore Developer Documentation. https://developers.google.com/ar
4. Unity Technologies. Input System Package Documentation. https://docs.unity3d.com/Packages/com.unity.inputsystem@1.14/manual/index.html
5. Unity Technologies. Universal Render Pipeline Documentation. https://docs.unity3d.com/Packages/com.unity.render-pipelines.universal@17.0/manual/index.html
6. Unity Technologies. Physics.SphereCast and Continuous Collision Detection, Unity Manual and Scripting Reference. https://docs.unity3d.com/ScriptReference/Physics.SphereCast.html
7. Unity Technologies. TextMeshPro Documentation. https://docs.unity3d.com/Packages/com.unity.textmeshpro@3.0/manual/index.html
8. Project source code, CLAUDE.md rules file and Git history of the AR Target Shooter project, authored by the student.

Certificate

